

```
{
  "creator" : "ari.neko@cvut.cz",
  "timestamp" : "09-07-2026_17:23:50",
  "variant" : 2,
  "subjectName" : "java-programming",
  "categories" : [ "Generics & Types", "Java Collections Framework", "Object-Oriented Principles" ],
  "questions" : [ {
    "question" : "Why are primitive types like int or char forbidden from being passed directly as parameters to Java Generics?",
    "answers" : [ "Primitives consume too much L1 cache memory causing thread scheduler stalls", "Generics require object references on the heap; primitives do not inherit from Object", "Database drivers cannot bind primitive types to statement query parameters", "The modularity guidelines written in module-info block primitive parameters" ],
    "correctIndex" : 1
  }, {
    "question" : "What primary objective does Java's Type Erasure accomplish when compiling generic code architectures?",
    "answers" : [ "It restricts collection instances from scaling horizontally", "It converts object reference addresses into encrypted byte arrays", "It forces the compiler to duplicate class files for every unique type", "It replaces type parameters with bounds or Object to ensure type safety without runtime overhead" ],
    "correctIndex" : 3
  }, {
    "question" : "What behavioral trait separates a TreeMap from a standard HashMap implementation?",
    "answers" : [ "TreeMap is limited to storing integer primitive values", "TreeMap maintains its keys sorted according to their natural ordering or a custom Comparator", "HashMap encrypts key fields; TreeMap leaves characters visible", "TreeMap prevents asynchronous thread execution steps from accessing data" ],
    "correctIndex" : 1
  }, {
    "question" : "Which interface inside the Java Collections Framework guarantees that no duplicate elements can be stored?",
    "answers" : [ "Map entries tracker", "List", "Set", "Queue" ],
    "correctIndex" : 2
  }, {
    "question" : "How does a HashMap handle key value lookups efficiently under the hood?",
    "answers" : [ "It computes the key's hashCode() to locate a bucket array slot containing data node links", "It writes the object properties out to temporary text files", "It references a foreign key constraint row mapped from a local tracker", "It runs a full linear search loop across all entries sequentially" ],
    "correctIndex" : 0
  }
}
```

```

}, {
"question" : "What is a major performance difference between an ArrayList and a LinkedList in Java?",
"answers" : [ "ArrayList resizes dynamically without duplicating its internal index arrays",
"LinkedList consumes less memory because it drops link node pointer sets", "ArrayList provides O(1) random access time; LinkedList provides O(1) element insertion at boundaries", "LinkedList supports horizontal clustering distributions" ],
"correctIndex" : 2
}, {
"question" : "What restriction applies to a class declared with the final modifier keyword in Java?",
"answers" : [ "The internal variables are rendered immutable values instantly", "The class cannot be extended or subclassed by any other class structure", "The JVM forces code tasks to run inside kernel mode", "The class cannot contain any public instance initialization constructors" ],
"correctIndex" : 1
}, {
"question" : "What does a class inherit when it implements a Java Interface containing default methods?",
"answers" : [ "The concrete private instance fields", "Complete access to the abstract module's private memory", "The interface's default functional implementations without breaking backward compatibility", "Automated thread synchronization parameters across loop executions" ],
"correctIndex" : 2
}, {
"question" : "Which pillar of OOP allows a subclass to provide a specific implementation of a method defined in its superclass?",
"answers" : [ "Polymorphism (Method Overriding)", "Abstraction execution rules", "Inheritance (Method Overloading)", "Encapsulation" ],
"correctIndex" : 0
}, {
"question" : "What separates method overriding from method overloading in Java programming execution?",
"answers" : [ "Overriding replaces a superclass method at runtime; overloading uses distinct parameters at compile-time", "Overloading is exclusive to abstract interfaces", "Overloading forces garbage collection routines", "Overriding modifies class access modifiers; overloading alters return type bounds" ],
"correctIndex" : 0
}]
}

```